Experiment No.04

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance.

Theory:

1. Dataset Source

Dataset: Iris
Source: Kaggle

Link: https://www.kaggle.com/datasets/uciml/iris

2. Dataset Description

The Iris dataset contains measurements of iris flowers belonging to three different species. The goal is to classify the species based on flower measurements.

```
df = pd.read_csv("sample_data/Iris.csv")

df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN) is a supervised learning algorithm used for classification and regression.

It works by identifying the K closest data points to a given input sample and assigning the most frequent class among them.

Distance Calculation

The most commonly used distance metric is Euclidean Distance.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Classification Rule

The predicted class is determined by majority voting among the nearest neighbors.

$$\hat{y} = \text{mode}(y_1, y_2, ..., y_k)$$

4. Algorithm Limitations

Although KNN is simple and effective, it has several limitations:

- High computational cost for large datasets
- Performance decreases with high-dimensional data
- Sensitive to irrelevant features
- Requires feature scaling
- Prediction time is slower because it stores the entire training dataset
- Choosing the optimal K value can be difficult

5. Methodology / Workflow

The following steps were followed to implement the experiment.

Step 1: Dataset Collection

The Iris dataset was downloaded from Kaggle.

Step 2: Data Loading

The dataset was loaded using the Pandas library in Python.

Step 3: Data Preprocessing

- Checked dataset structure
- Verified data types and summary statistics
- Separated input features and target variable

Step 4: Feature Scaling

Since KNN relies on distance calculations, StandardScaler was used to normalize feature values.

Step 5: Train-Test Split

The dataset was divided into:

- Training Data (80%)
- Testing Data (20%)

Step 6: Model Training

The K-Nearest Neighbors classifier was implemented using the training dataset.

Step 7: Prediction

The trained model predicted the species of flowers in the test dataset.

Step 8: Model Evaluation

Model performance was evaluated using:

- Accuracy Score
- Confusion Matrix

Step 9: Visualization

Graphs were generated to analyze:

- Accuracy of the model

6. Performance Analysis

Model performance was evaluated using accuracy and confusion matrix.

Accuracy Formula

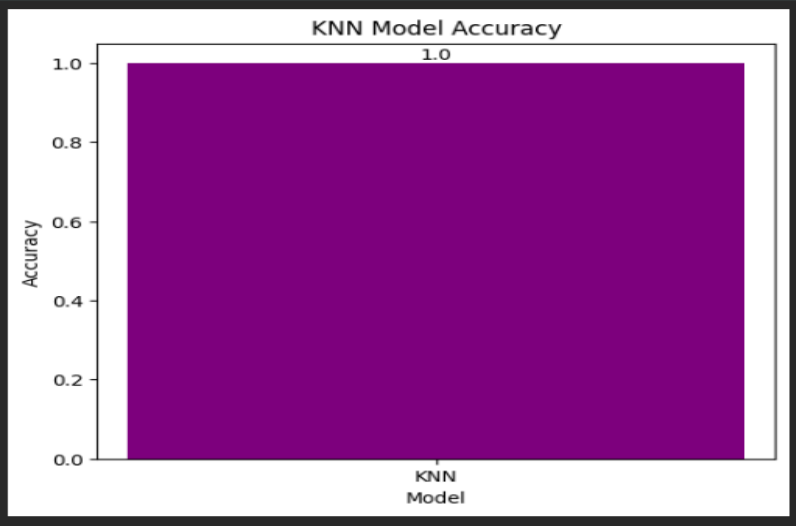
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

```python
accuracy = accuracy_score(y_test, y_pred)

accuracy_df = pd.DataFrame({
    "Model": ["KNN"],
    "Accuracy": [accuracy]
})

accuracy_df
```

	Model	Accuracy
0	KNN	1.0



7. Hyperparameter Tuning

Hyperparameter tuning was performed to determine the optimal value of K (number of neighbors).

Parameter Tuned

- n_neighbors (K value)

Different values of K were tested:

K = 1 to 20

Process

For each K value:

1. Train the KNN model
2. Predict the test dataset
3. Calculate accuracy
4. Plot Error Rate vs K Value

The graph shows the relationship between K value and model error rate.

- Lower error rate indicates better performance
- The optimal K value was observed around K = 3

Source Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

#Load Dataset

df = pd.read_csv("Iris.csv")

df.head()

#Dataset Overview

df.info()

df.describe()

#Define Features and Target

X = df.drop("Species", axis=1)

y = df["Species"]

X.head()

#Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)

X_train.head()

#Feature Scaling

KNN works based on distance calculation, so scaling is important.

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

pd.DataFrame(X_train).head()
```

```
#Train KNN Model

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

y_pred = knn_model.predict(X_test)

pd.DataFrame(y_pred, columns=["Predicted Species"]).head()

#Accuracy Evaluation

accuracy = accuracy_score(y_test, y_pred)

accuracy_df = pd.DataFrame({

   "Model": ["KNN"],

   "Accuracy": [accuracy]

})

accuracy_df


#Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix - KNN")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

Output:

```
df.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
X = df.drop("Species", axis=1)
y = df["Species"]

X.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	1	5.1	3.5	1.4	0.2
1	2	4.9	3.0	1.4	0.2
2	3	4.7	3.2	1.3	0.2
3	4	4.6	3.1	1.5	0.2
4	5	5.0	3.6	1.4	0.2

Next steps: Generate code with X | New interactive sheet

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
22	23	4.6	3.6	1.0	0.2
15	16	5.7	4.4	1.5	0.4
65	66	6.7	3.1	4.4	1.4
11	12	4.8	3.4	1.6	0.2
42	43	4.4	3.2	1.3	0.2

Next steps: Generate code with X_train | New interactive sheet

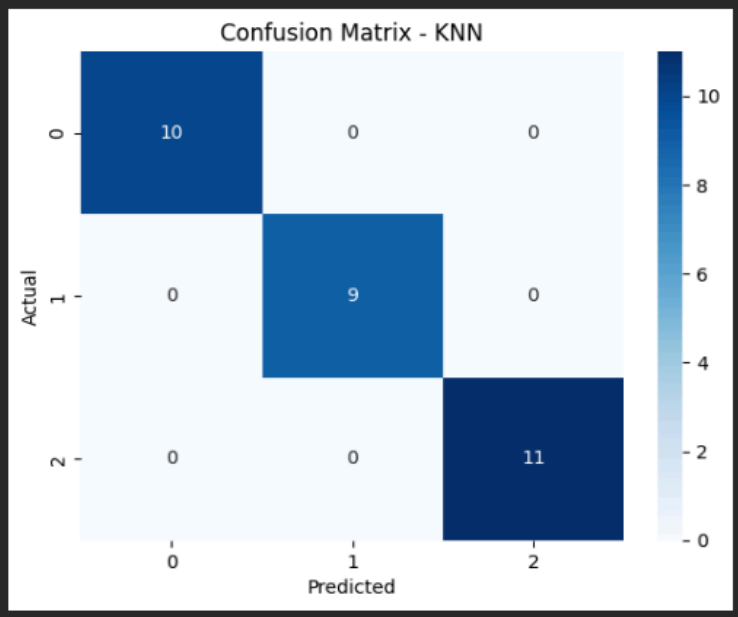
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

pd.DataFrame(X_train).head()
```

	0	1	2	3	4
0	-1.210307	-1.473937	1.220379	-1.563987	-1.309484
1	-1.372402	-0.133071	3.020017	-1.277280	-1.042922
2	-0.214583	1.085898	0.095606	0.385621	0.289886
3	-1.465027	-1.230143	0.770470	-1.219939	-1.309484
4	-0.747179	-1.717731	0.320560	-1.391963	-1.309484

```
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

y_pred = knn_model.predict(X_test)

pd.DataFrame(y_pred, columns=["Predicted Species"]).head()
```

	Predicted Species
0	Iris-versicolor
1	Iris-setosa
2	Iris-virginica
3	Iris-versicolor
4	Iris-versicolor

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Conclusion:

In this experiment, the K-Nearest Neighbors algorithm was implemented on the Iris dataset for classification. The model achieved high accuracy in predicting iris flower species based on sepal and petal measurements. Hyperparameter tuning helped determine the optimal value of K, improving the model's classification performance. The results demonstrate that KNN is an effective algorithm for small and well-structured classification datasets.